SISTEMAS DISTRIBUIDOS

Apache Kafka

com Java

Atividade Pratica — Produção e Consumo de Eventos

Compreender a arquitetura do Kafka	**Implementar Producer e Consumer em Java**	**Aplicar garantias de entrega em um cenário real**

1. Introdução

O Apache Kafka e uma plataforma distribuída de streaming de eventos, desenvolvida originalmente pelo LinkedIn em 2011 para resolver o problema de ingestão de grandes volumes de dados de atividade de usuários e, posteriormente, doada para a Apache Software Foundation. Diferentemente de um broker de mensagens tradicional — orientado ao transporte e descarte de mensagens após o consumo —, o Kafka foi projetado como um log distribuído, imutável e particionado de eventos, com foco em alta vazão, baixa latência e replicação tolerante a falhas.

O Kafka é adotado em produção por organizações como Netflix, Uber, Airbnb e mais de 80% das empresas da Fortune 100. Seus principais casos de uso incluem pipelines de dados em tempo real, integração de microsserviços por eventos (Event-Driven Architecture), auditoria de ações, processamento de streams e replicação de estados entre sistemas.

Esta atividade tem como objetivo capacitar o discente a configurar um ambiente Kafka local com Docker, compreender seus conceitos fundamentais e implementar produtores e consumidores em Java utilizando a biblioteca oficial kafka-clients.

2. Conceitos Fundamentais

2.1 Evento (Event / Record)

A unidade atômica de informação no Kafka e o evento (também denominado record ou mensagem). Cada evento e composto pelos seguintes campos:

Campo	Descrição	Observação
Key	Chave opcional — array de bytes	Determina a partição de destino via hash murmur2; garante ordenação por entidade
Value	Conteúdo do evento — array de bytes	Geralmente JSON, Avro ou Protobuf
Timestamp	Instante de criação do evento	Atribuido pelo producer ou pelo broker, conforme configuração do tópico
Headers	Pares chave-valor de metadados	Ex: versão de schema, ID de correlação, tipo do evento

2.2 Tópico (Topic)

Um tópico e uma categoria logica de eventos — análogo a uma tabela em um banco de dados ou a um canal de comunicação. Os tópicos Kafka possuem retenção configurável (padrão: 7 dias); os eventos não são removidos apos o consumo e permanecem disponíveis para releitura por qualquer consumidor interessado. Essa característica distingue fundamentalmente o Kafka de um broker de mensagens tradicional e central para o reprocessamento de histórico.

2.3 Partição (Partition) e Offset

Cada tópico e subdividido em N partições. Uma partição e um log sequencial, imutável e estritamente ordenado de eventos. O offset e o numero sequencial que identifica a posição de cada evento dentro de uma partição (inicia em 0). O par (partição, offset) identifica univocamente qualquer evento no cluster.

Consumidores armazenam o offset do ultimo evento processado e podem retomar a leitura de qualquer ponto, inclusive do inicio. Essa capacidade e o fundamento do reprocessamento e da tolerância a falhas no Kafka.

Ordenação: O Kafka garante ordenação estrita dentro de uma partição, mas não entre partições distintas. Por isso, ao utilizar o código de uma entidade (Ex: ID de pedido) como Key, todos os eventos dessa entidade são roteados para a mesma partição, assegurando ordenacao causal entre eles.

2.4 Producer

O Producer e o componente responsável por publicar eventos em tópicos. A partição de destino e determinada pelo hash murmur2 da Key. Eventos sem Key (null) são distribuídos por round-robin entre as partições disponíveis. O producer acumula eventos em um buffer interno e os envia em lotes (batches) ao broker para maximizar o throughput.

2.5 Consumer e Consumer Group

O Consumer lê eventos de tópicos a partir de um offset especifico. Consumidores se organizam em Consumer Groups identificados por um group.id. O Kafka distribui as partições do tópico entre os membros do grupo, garantindo que cada partição seja consumida por exatamente um membro em um dado momento. Diferentes Consumer Groups são totalmente independentes entre si: cada um mantem seu próprio conjunto de offsets e consome o tópico sem interferência mútua.

Configuracao	Comportamento	Caso de uso tipico
1 consumer, 3 partições	1 instancia consome as 3 partições	Desenvolvimento, volume baixo
3 consumers, 3 partições	1 particao por consumer — paralelismo maximo	Producao — escalabilidade horizontal
4 consumers, 3 partições	1 consumer ficara ocioso	Nao ha ganho alem de N = numero de partições

2.6 Broker, Cluster e KRaft

Um broker e uma instancia do servidor Kafka. Um cluster Kafka e formado por multiplos brokers, cada um armazenando um subconjunto das partições dos tópicos. A coordenacao do cluster era historicamente delegada ao Apache ZooKeeper; a partir do Kafka 3.3, o modo KRaft (Kafka Raft Metadata) elimina essa dependencia externa e utiliza o algoritmo de consenso Raft para coordenacao nativa. Os exemplos desta atividade utilizam o modo KRaft.

2.7 Garantias de Entrega

O Kafka oferece tres niveis de garantia de entrega, configurados por propriedades do producer e do consumer. A escolha envolve uma troca (trade-off) entre durabilidade, throughput e complexidade de implementacao:

Garantia	Semantica	Configuracao Producer	Configuracao Consumer
At-most-once	Pode perder eventos; nunca duplica	acks=0	enable.auto.commit=true
At-least-once	Nunca perde; pode duplicar em falhas	acks=all	commitSync() manual

Garantia	Semantica	Configuracao Producer	Configuracao Consumer
Exactly-once	Sem perda e sem duplicacao garantidos	acks=all + idempotence=true	Transacoes Kafka

Recomendacao: Para a maioria das aplicacoes de negocio, at-least-once com idempotencia no processamento (ou seja, processar o mesmo evento duas vezes não causa efeito colateral) e o balanco mais pragmatico entre seguranca e desempenho.

2.8 Topico Interno __consumer_offsets

O Kafka armazena os offsets confirmados por cada Consumer Group em um tópico interno especial chamado __consumer_offsets. Ao chamar commitSync() ou commitAsync(), o consumer grava nesse tópico o proximo offset a ser lido. Em caso de reinicializacao, o consumer consulta esse tópico para retomar do ponto correto — sem necessidade de nenhum armazenamento externo.

3. Configuracao do Ambiente com Docker

Para esta atividade, utiliza-se o Docker Compose a seguir, que sobe o Kafka em modo KRaft (sem ZooKeeper) e a interface grafica Kafka-UI para inspecao do cluster em tempo real. Nenhuma outra dependencia de infraestrutura e necessaria.

```
# docker-compose.yml
version: '3.8'
services:
  kafka:
    image: confluentinc/cp-kafka:7.6.0
    hostname: kafka
    ports:
      - "9092:9092"                                    // porta de acesso para as
aplicacoes Java no host
    environment:
      KAFKA_NODE_ID: 1                                 // ID unico deste no no cluster
      KAFKA_PROCESS_ROLES: 'broker,controller'         // KRaft: no atua como broker E
como controlador
      KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:29093' // quorum de controle:
nodeId@host:porta
      KAFKA_LISTENERS: 'PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:29093'//
listeners internos
      KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://localhost:9092'// endereco anunciado
aos clientes Java
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
'PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT'// protocolo por listener
      KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'    // listener exclusivo para
controle do cluster
      KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'    // listener para comunicação
entre brokers
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'          // tópicos criados
automaticamente na primeira publicacao
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1        // fator 1 adequado para
ambiente local (producao: 3)
      CLUSTER_ID: 'MkU3OEVBNTcwNTJENDM2Qk'             // ID fixo — garante
persistencia entre reinicializacoes
  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    ports:
      - "8080:8080"                                    // interface web de
gerenciamento do cluster
    environment:
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: 'kafka:9092' // usa o hostname Docker, não
'localhost'
    depends_on: [kafka]
```

Analise do codigo — Docker Compose — decisoes de configuracao

1. KAFKA_PROCESS_ROLES: 'broker,controller' configura um no unico para atuar como broker (processa mensagens dos clientes) e como controller (coordena o cluster via KRaft). Em producao, essas funcoes são separadas em nos distintos para maior resiliencia.

2. KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://localhost:9092' e o endereco que o broker anuncia aos clientes. Aplicacoes Java rodando no host Docker devem se conectar a 'localhost:9092'. Aplicacoes rodando dentro de outros containers devem usar 'kafka:9092' (hostname do servico).

3. KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1 adequa o tópico interno __consumer_offsets a um cluster de um unico broker. Em clusters de producao com 3 ou mais brokers, o valor recomendado e 3 para tolerancia a falhas.

4. CLUSTER_ID: valor fixo e imutavel que identifica o cluster. Deve ser gerado uma unica vez (ex: via kafka-storage random-uuid) e reutilizado. Um ID diferente a cada inicializacao do container apaga o histórico de tópicos e offsets do cluster.

Instrucao de inicializacao: Execute 'docker compose up -d' no diretorio do arquivo docker-compose.yml. Aguarde o container estabilizar (cerca de 15 segundos) e acesse a Kafka-UI em http://localhost:8080 para confirmar que o broker esta ativo.

4. Configuracao do Projeto Maven

Os exemplos de codigo desta atividade utilizam a biblioteca oficial kafka-clients, mantida pelo projeto Apache Kafka. Deve-se criar um projeto Maven com a seguinte dependencia no arquivo pom.xml:

```
<!-- pom.xml -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>br.edu.disciplina</groupId>
  <artifactId>kafka-atividade</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>

    <!-- Biblioteca oficial do Apache Kafka para Java -->// inclui Producer, Consumer
e Admin Client
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
      <version>3.7.0</version>                        // versao estavel LTS em 2025
    </dependency>

    <!-- Logging -->                                  // necessario para suprimir
avisos de slf4j
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>2.0.13</version>
    </dependency>

  </dependencies>
</project>
```

Estrutura de pacotes sugerida: Organizar o projeto em tres pacotes: kafka.producer (classes de producao de eventos), kafka.consumer (classes de consumo) e kafka.model (POJOs de dominio e utilitarios de serializacao JSON).

5. Implementacao do Producer

5.1 Funcionamento Interno do Producer

O Producer Java mantem um buffer interno de registros (RecordAccumulator) organizado por particao de destino. Registros são acumulados em lotes (batches) ate que um dos seguintes criterios seja satisfeito: o lote atingiu o tamanho maximo (batch.size, padrão 16 KB), o tempo de espera expirou (linger.ms, padrão 0 ms) ou flush() foi chamado explicitamente. Somente entao o lote e transmitido ao broker. Esse mecanismo maximiza o throughput ao reduzir o numero de requisicoes de rede.

Apos o envio, o broker aguarda a confirmacao das replicas conforme configurado em acks e retorna os metadados do registro (particao e offset atribuidos) ou um erro. O Producer pode retentar automaticamente erros transientes (retries) antes de propagar a falha ao codigo da aplicacao.

5.2 Propriedades Essenciais do Producer

Propriedade	Valor tipico	Descricao
bootstrap.servers	host:porta	Lista inicial de brokers para descoberta do cluster. O cliente descobre os demais automaticamente.
key.serializer	StringSerializer	Converte a Key para bytes antes do envio ao broker.
value.serializer	StringSerializer	Converte o Value (ex: JSON) para bytes antes do envio.
acks	all	Nivel de confirmacao: 0=nenhum, 1=lider, all=todos os replicas sincronos (ISR).
enable.idempotence	true	Previne duplicatas em retransmissoes. Exige acks=all e retries > 0.
retries	3	Numero de retentativas automaticas em erros transientes.
linger.ms	5	Tempo de espera para acumular registros em um lote antes do envio. Aumenta latencia, melhora throughput.
batch.size	16384	Tamanho maximo em bytes de um lote por particao.

5.3 Exemplo Completo — Producer com Callback

O exemplo a seguir implementa um Producer que publica eventos de rastreamento de encomendas. A utilizacao de Keys garante que todos os eventos de uma mesma encomenda sejam roteados para a mesma particao, preservando a ordenacao causal. O callback assincrono permite monitorar a confirmacao de cada evento sem bloquear a thread principal:

```
package kafka.producer;
```

```
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.clients.producer.RecordMetadata;
import java.util.Properties;
import java.time.Instant;

public class EventoEncomendaProducer {

    private static final String TOPICO = "logfast.eventos.encomenda";// nome do
tópico de destino

    public static void main(String[] args) throws InterruptedException {

        Properties props = new Properties();           // (1) mapa de configuracoes

        props.put("bootstrap.servers", "localhost:9092");// ponto de entrada para
descoberta do cluster
        props.put("key.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");// Key
(String) -> bytes
        props.put("value.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");// Value (JSON
String) -> bytes

        props.put("acks", "all");                      // aguarda confirmacao de todos
os replicas sincronos (ISR)
        props.put("enable.idempotence", "true");       // previne duplicatas em
retransmissoes de rede
        props.put("retries", "3");                     // tentativas automaticas em
erros transientes
        props.put("linger.ms", "5");                   // aguarda 5ms para agrupar
registros em lotes maiores

        try (var producer = new KafkaProducer<String, String>(props)) {// (2) try-
with-resources fecha o producer ao final

            String[] encomendas = {"ENC-001", "ENC-002", "ENC-003", "ENC-004",
"ENC-005"};
            String[] statusList = {"COLETADO", "TRIAGEM", "EM_TRANSITO",
"ENTREGUE"};

            for (String encId : encomendas) {
                for (String status : statusList) {

                    String key = encId;                // (3) Key = ID da encomenda ->
mesma particao, ordenacao garantida
                    String value = String.format(
                        "{"encomendaId":"%s","status":"%s","timestamp":"%s"}",
                        encId, status, Instant.now());

                    var record = new ProducerRecord<>(TOPICO, key, value);// (4)
encapsula tópico, key e value

                    producer.send(record, EventoEncomendaProducer::onEnvio);// (5)
envio assincrono com callback
```

```
                    Thread.sleep(50); // simula intervalo entre eventos
                }
            }

            producer.flush();                          // (6) aguarda confirmacao de
todos os registros pendentes no buffer
            System.out.println("Todos os eventos publicados.");

        } // (7) producer.close() chamado automaticamente pelo try-with-resources
    }

    private static void onEnvio(RecordMetadata meta, Exception ex) {// (8) callback:
invocado apos resposta do broker
        if (ex != null) {
            System.err.println("Falha ao publicar: " + ex.getMessage());
            return;
        }
        System.out.printf("Publicado -> tópico=%-30s part=%d off=%d%n",
            meta.topic(), meta.partition(), meta.offset());
    }
}
```

Analise do codigo — Producer — analise detalhada

1. bootstrap.servers: lista inicial de brokers para conexao. O cliente Kafka descobre automaticamente os demais brokers do cluster a partir dessas entradas. Em producao, recomenda-se listar pelo menos 3 brokers distintos para tolerancia a falhas na inicializacao.

2. acks=all (equivalente a -1): o producer so considera o envio confirmado apos todos os replicas sincronos (In-Sync Replicas — ISR) terem persistido a mensagem em seus logs. E a configuracao mais segura, a custo de alguma latencia adicional.

3. enable.idempotence=true: cada producer recebe um Producer ID (PID) e numera seus registros sequencialmente por particao. O broker rejeita registros com o mesmo PID e numero de sequencia, garantindo que retransmissoes causadas por timeout ou falha de rede não gerem eventos duplicados no log.

4. A Key determina a particao de destino via hash murmur2. Ao usar o ID da encomenda como Key, todos os eventos de ENC-001 sempre vao para a mesma particao, garantindo que um Consumer lendo essa particao os veja em ordem cronologica.

5. O callback onEnvio e invocado em uma thread interna do producer (I/O thread), não na thread principal. Por isso, operacoes de UI ou acesso a estado compartilhado dentro do callback devem ser sincronizadas.

6. producer.flush() difere de producer.close(). O flush() bloqueia ate que todos os eventos no buffer interno sejam enviados e confirmados pelo broker, mas mantem o producer reutilizavel. O close() realiza o flush e depois encerra o producer definitivamente.

6. Implementacao do Consumer

6.1 Funcionamento Interno do Consumer

O Consumer Java opera em um loop de polling: a cada iteracao, o metodo poll() solicita um lote de registros ao broker e retorna imediatamente caso não haja eventos novos ou apos o tempo de espera configurado. O broker usa a frequencia das chamadas a poll() como mecanismo de deteccao de vida (heartbeat implicito): se o consumer não chamar poll() dentro de max.poll.interval.ms (padrão: 5 minutos), o broker o considera morto, remove-o do grupo e aciona um rebalanceamento das partições.

O rebalanceamento e o processo pelo qual o Kafka redistribui as partições entre os membros do Consumer Group. Ele ocorre quando: um novo consumer entra no grupo, um consumer sai ou falha, ou a assinatura de tópicos e alterada. Durante o rebalanceamento, nenhum consumer do grupo processa eventos (stop-the-world).

6.2 Modos de Commit de Offset

O commit de offset e o mecanismo pelo qual o consumer informa ao broker o progresso do processamento. A escolha do modo impacta diretamente as garantias de entrega:

Modo	Como funciona	Garantia	Risco
Auto-commit	Commit automatico a cada auto.commit.interval.ms	At-most-once	Perde eventos se a aplicacao falhar apos o commit automatico e antes de concluir o processamento
commitSync()	Commit sincrono — bloqueia ate a confirmacao do broker	At-least-once	Pode reprocessar eventos em caso de falha antes do commit
commitAsync()	Commit assincrono — não bloqueia a thread	At-least-once	Pode reprocessar eventos; requer callback para tratamento de erros
Commit por offset	commitSync(offsets) para partições especificas	At-least-once com granularidade fina	Complexidade de implementacao maior

6.3 Propriedades Essenciais do Consumer

Propriedade	Valor tipico	Descricao
bootstrap.servers	host:porta	Lista inicial de brokers para descoberta do cluster.
group.id	grupo-app	Identificador do Consumer Group. Determina como as partições são distribuidas entre os consumers.
key.deserializer	StringDeserializer	Converte bytes recebidos do broker de volta para o tipo da Key.
value.deserializer	StringDeserializer	Converte bytes recebidos do broker de volta para o tipo do Value.
auto.offset.reset	earliest	Comportamento ao não encontrar offset gravado: 'earliest' inicia do offset 0; 'latest' inicia apenas dos eventos novos.
enable.auto.commit	false	Desabilitar para usar commitSync() manual e ter controle explícito sobre at-least-once.
max.poll.records	500	Numero maximo de registros retornados por poll(). Ajustar conforme a capacidade de processamento da aplicacao.

6.4 Exemplo Completo — Consumer com Commit Manual

O exemplo a seguir implementa um Consumer que mantem o estado atual de cada encomenda em memoria. O commit manual garante que o offset so seja confirmado apos o processamento bem-sucedido de cada lote, implementando a garantia at-least-once de forma explicita:

```
package kafka.consumer;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import java.time.Duration;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Properties;

public class RastreamentoConsumer {

    private static final String TOPICO  = "logfast.eventos.encomenda";// tópico a
consumir
    private static final String GROUP   = "grupo-rastreamento";// ID do Consumer
Group

    public static void main(String[] args) {

        Properties props = new Properties();           // (1) configuracoes do consumer
        props.put("bootstrap.servers",  "localhost:9092");
        props.put("group.id",            GROUP);       // CRITICO: identifica este
consumer group no cluster
```

```
        props.put("key.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset",   "earliest"); // sem offset gravado: ler desde
o inicio do tópico
        props.put("enable.auto.commit",  "false");    // commit manual via
commitSync()
        props.put("max.poll.records",    "100");      // maximo de registros por lote

        Map<String, String> estadoAtual = new HashMap<>();// (2) mapa em memoria:
encomendaId -> status atual

        try (var consumer = new KafkaConsumer<String, String>(props)) {

            consumer.subscribe(List.of(TOPICO));      // (3) inscreve no tópico;
broker distribui partições ao grupo
            System.out.println("Consumer iniciado. Aguardando eventos...");

            while (true) {                            // (4) loop de polling — padrão
de consumo no Kafka

                ConsumerRecords<String, String> lote =// (5) solicita lote ao broker
                    consumer.poll(Duration.ofMillis(300));// bloqueia ate 300ms
esperando novos eventos

                for (ConsumerRecord<String, String> r : lote) {
                    processar(r, estadoAtual);
                }

                if (!lote.isEmpty()) {                // (6) commit somente apos
processar o lote inteiro
                    consumer.commitSync();            // grava offset em
__consumer_offsets; bloqueia ate confirmar
                    imprimirEstado(estadoAtual);
                }
            }
        }
    }

    private static void processar(
            ConsumerRecord<String, String> r,
            Map<String, String> estado) {
        // extrai o status do payload JSON (simplificado)
        String json   = r.value();
        int idx       = json.indexOf(""status":"") + 10;
        String status = json.substring(idx, json.indexOf("""", idx));

        estado.put(r.key(), status);                  // atualiza o status mais
recente da encomenda
        System.out.printf("[part=%d off=%4d] %-10s -> %s%n",
            r.partition(), r.offset(), r.key(), status);
    }

    private static void imprimirEstado(Map<String, String> estado) {
```

```
        System.out.println("--- Estado atual das encomendas ---");
        estado.forEach((id, s) -> System.out.printf("  %-10s -> %s%n", id, s));
    }
}
```

Analise do codigo — Consumer — analise detalhada

1. group.id e a propriedade mais critica do consumer. Todos os consumers com o mesmo group.id formam um Consumer Group. O Kafka distribui as partições entre os membros do grupo, garantindo que cada particao seja lida por exatamente um membro em um dado momento. Ao subir 3 instancias deste consumer (com o mesmo group.id e 3 partições no tópico), cada instancia consumira 1 particao de forma paralela.

2. auto.offset.reset=earliest: ao não encontrar offset gravado para o group.id (primeira execucao, ou grupo deletado), o consumer inicia a leitura do offset 0 de cada particao. O valor 'latest' e mais comum em producao — inicia apenas dos eventos publicados apos a inscricao do consumer.

3. enable.auto.commit=false com commitSync(): o offset e gravado em __consumer_offsets somente apos o bloco de processamento concluir sem excecao. Se a aplicacao falhar antes do commitSync(), o mesmo lote sera reprocessado na proxima execucao — garantia at-least-once. Com auto.commit=true, o offset e confirmado periodicamente de forma independente do processamento, o que pode gerar perda silenciosa de eventos.

4. consumer.poll() deve ser chamado regularmente. O broker usa o intervalo entre chamadas para detectar consumers mortos via heartbeat implicito. Se poll() não for chamado dentro de max.poll.interval.ms (padrão: 5 minutos), o broker remove o consumer do grupo e aciona rebalanceamento das partições.

5. Em producao, o bloco de processamento deve estar envolto em try/catch com uma estrategia definida de tratamento de erro: N tentativas de reprocessamento, seguidas de publicacao no tópico de Dead Letter Queue (DLQ) e commitSync() para que o consumer avance para o proximo evento.

6.5 Exemplo — Dois Consumer Groups Independentes

Um mesmo tópico pode ser consumido simultaneamente por multiplos Consumer Groups de forma completamente independente, cada um mantendo seu proprio conjunto de offsets. O exemplo a seguir implementa um segundo grupo dedicado a persistencia em CSV — a execucao paralela dos dois consumers demonstra o isolamento garantido pelo Kafka:

```
package kafka.consumer;

import org.apache.kafka.clients.consumer.KafkaConsumer;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.time.Duration;
import java.util.List;
import java.util.Properties;

public class AnalyticsConsumer {

    private static final String GROUP = "grupo-analytics";// group.id DIFERENTE do
RastreamentoConsumer

    public static void main(String[] args) throws Exception {
```

```
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id",           GROUP);       // grupo independente: offset
proprio, sem interferencia
        props.put("key.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset",  "earliest");
        props.put("enable.auto.commit", "false");

        try (var consumer = new KafkaConsumer<String, String>(props);
             var writer = new PrintWriter(new FileWriter("eventos.csv", true))) {//
abre (ou cria) o CSV em modo append

            writer.println("encomendaId,status,timestamp");// cabecalho do CSV
            consumer.subscribe(List.of("logfast.eventos.encomenda"));

            while (true) {
                var lote = consumer.poll(Duration.ofMillis(300));

                for (var r : lote) {
                    String[] campos = extrairCampos(r.value());// parseia o JSON do
Value
                    writer.printf("%s,%s,%s%n",       // grava linha no CSV
                        campos[0], campos[1], campos[2]);
                }

                if (!lote.isEmpty()) {
                    writer.flush();                   // persiste o CSV antes do
commit
                    consumer.commitSync();            // confirma offset somente apos
persistencia bem-sucedida
                }
            }
        }
    }

    private static String[] extrairCampos(String json) {
        // extracao simplificada; usar Jackson em producao
        String id = extrair(json, "encomendaId");
        String st = extrair(json, "status");
        String ts = extrair(json, "timestamp");
        return new String[]{id, st, ts};
    }
    private static String extrair(String json, String campo) {
        int i = json.indexOf("""" + campo + "":"") + campo.length() + 4;
        return json.substring(i, json.indexOf("""", i));
    }
}
```

Analise do codigo — Consumer Groups independentes — ponto critico

1. O group.id 'grupo-analytics' e distinto de 'grupo-rastreamento'. O Kafka mantem offsets separados para cada grupo no tópico __consumer_offsets. Os dois consumers podem ser executados simultaneamente a partir do mesmo tópico sem que um interfira no progresso do outro.
2. O writer.flush() e chamado antes do consumer.commitSync(). Essa ordem e deliberada: garante que os dados estejam gravados em disco antes que o offset seja confirmado. Se a aplicacao falhar apos o flush e antes do commit, o evento sera reprocessado e a linha gravada novamente — isso exige que a aplicacao trate duplicatas (ex: verificar se a linha ja existe).
3. Ao executar os dois consumers simultaneamente (em terminais distintos) e verificar na Kafka-UI a aba 'Consumer Groups', sera possivel observar que cada grupo mantem lag e offsets independentes para cada particao do tópico.

7. Tratamento de Falhas e Dead Letter Queue

7.1 O Problema de Eventos Inprocessaveis

Em sistemas de producao, e inevitavel que determinados eventos causem falhas no processamento: payloads malformados, violacoes de regras de negocio, dependencias externas indisponiveis. Sem uma estrategia de tratamento, o consumer pode entrar em um loop infinito de reprocessamento, bloqueando todos os eventos subsequentes na mesma particao (poison pill).

A solucao padrão no ecossistema Kafka e a Dead Letter Queue (DLQ): um tópico especifico para o qual eventos inprocessaveis são encaminhados apos N tentativas fracassadas. Isso permite que o fluxo principal avance enquanto os eventos problematicos ficam disponíveis para analise e reprocessamento manual posterior.

7.2 Implementacao de Retry com DLQ

O exemplo a seguir estende o AnalyticsConsumer com logica de retry (maximo 3 tentativas) e encaminhamento para um tópico DLQ apos o esgotamento das tentativas. Observe que o Producer e instanciado dentro do Consumer para publicar na DLQ:

```
package kafka.consumer;

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.clients.producer.*;
import java.time.Duration;
import java.util.*;

public class ConsumerComDLQ {

    private static final int    MAX_TENTATIVAS = 3;    // numero maximo de tentativas
antes da DLQ
    private static final String TOPICO_DLQ    = "logfast.dlq";// tópico de Dead
Letter Queue

    public static void main(String[] args) {

        Properties propsConsumer = new Properties();
        propsConsumer.put("bootstrap.servers", "localhost:9092");
        propsConsumer.put("group.id",          "grupo-dlq-demo");
        propsConsumer.put("key.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        propsConsumer.put("value.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        propsConsumer.put("auto.offset.reset",  "earliest");
        propsConsumer.put("enable.auto.commit", "false");

        Properties propsProducer = new Properties();  // producer para publicar na DLQ
        propsProducer.put("bootstrap.servers", "localhost:9092");
        propsProducer.put("key.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");
```

```
        propsProducer.put("value.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");

        try (var consumer = new KafkaConsumer<String, String>(propsConsumer);
             var dlqProducer = new KafkaProducer<String, String>(propsProducer)) {

            consumer.subscribe(List.of("logfast.eventos.encomenda"));

            while (true) {
                var lote = consumer.poll(Duration.ofMillis(300));

                for (var record : lote) {
                    processarComRetry(record, dlqProducer);// tenta processar;
encaminha para DLQ se necessario
                }

                if (!lote.isEmpty()) consumer.commitSync();
            }
        }
    }

    private static void processarComRetry(
            ConsumerRecord<String, String> record,
            KafkaProducer<String, String> dlqProducer) {

        int tentativa = 0;
        while (tentativa < MAX_TENTATIVAS) {
            try {
                processarEvento(record); // logica de negocio
                return;                  // sucesso: encerra as tentativas// sai do
loop apos processamento bem-sucedido
            } catch (Exception ex) {
                tentativa++;
                System.err.printf("Tentativa %d/%d falhou para key=%s: %s%n",
                    tentativa, MAX_TENTATIVAS, record.key(), ex.getMessage());
            }
        }

        // Esgotou as tentativas — encaminha para a DLQ// preserva key e value
originais
        var dlqRecord = new ProducerRecord<>(TOPICO_DLQ, record.key(),
record.value());
        dlqRecord.headers().add("motivo", "MAX_TENTATIVAS_ATINGIDO".getBytes());//
metadado de diagnostico
        dlqRecord.headers().add("tópico-origem",      // rastreabilidade: tópico de
origem
            record.topic().getBytes());
        dlqProducer.send(dlqRecord);
        dlqProducer.flush();
        System.err.printf("Evento key=%s encaminhado para DLQ.%n", record.key());
    }

    private static void processarEvento(ConsumerRecord<String, String> r) {
        // Simula falha para eventos com status TRIAGEM// lancara excecao nestas
condicoes
```

```
        if (r.value().contains(""status":"TRIAGEM""))
            throw new RuntimeException("Status TRIAGEM não suportado neste
servico.");
        System.out.println("Processado: " + r.value());
    }
}
```

Analise do codigo — DLQ — pontos criticos

1. O tópico DLQ deve ser criado com retenção longa (ex: -1 = ilimitada ou 30 dias). Isso garante que eventos problematicos estejam disponíveis para analise e reprocessamento manual mesmo semanas apos o incidente.
2. Os headers adicionados ao registro DLQ (motivo, tópico-origem, tentativas) são metadados essenciais para o diagnóstico. Em producao, e comum tambem registrar timestamp da falha, mensagem de erro e stack trace serializado.
3. O commitSync() apos o lote e chamado mesmo que alguns eventos tenham sido encaminhados para a DLQ. Isso e intencional: o consumer deve avancar no offset para não reprocessar indefinidamente o mesmo lote. Os eventos problematicos ja foram preservados na DLQ.
4. Em sistemas de alta criticidade, o tópico DLQ deve ter pelo menos 2-3 replicas e um mecanismo de alerta automatico (ex: um consumer de monitoramento que verifica o lag da DLQ e dispara alertas caso eventos sejam publicados nela).

8. Cenario — Sistema de Rastreamento LogFast

Contexto do Problema

A empresa LogFast e uma startup de entrega expressa que esta modernizando sua infraestrutura de software. O sistema legado de rastreamento e monolitico e não comporta o volume crescente de operacoes. O desafio consiste em implementar um sistema de rastreamento de encomendas em tempo real, com suporte a multiplos consumidores independentes, tolerancia a falhas e histórico de eventos auditavel — utilizando exclusivamente Apache Kafka com Java.

8.1 Fluxo de Negocio

O ciclo de vida de uma encomenda na LogFast compreende cinco estagios sequenciais, cada um representado por um evento publicado no Kafka:

1. Coleta: o entregador registra a coleta via aplicativo movel.
2. Triagem: a encomenda chega ao centro de distribuicao e e classificada por rota.
3. Em transito: a encomenda esta em transporte entre centros de distribuicao.
4. Saiu para entrega: o veiculo de entrega saiu com a encomenda.
5. Entregue / Falha: conclusao do ciclo, com sucesso ou registro de falha operacional.

8.2 Requisitos do Sistema

Funcionais

- RF01: Cada mudanca de status deve gerar um evento publicado em tempo real no tópico Kafka.
- RF02: Um servico de rastreamento deve manter o ultimo status de cada encomenda em memoria.
- RF03: Um servico de analytics deve persistir todos os eventos em arquivo CSV para auditoria.
- RF04: Falhas no processamento devem acionar retry com encaminhamento para DLQ apos 3 tentativas.
- RF05: O histórico de eventos deve suportar reprocessamento completo a qualquer momento.

Nao Funcionais

- RNF01: Suporte a no minimo 500 eventos por segundo.
- RNF02: Garantia minima at-least-once para todos os eventos.
- RNF03: Retencao do histórico de eventos por no minimo 30 dias.
- RNF04: Os Consumer Groups devem operar de forma completamente independente.

8.3 Topologia Kafka

Topologia do Sistema LogFast

[App Movel/Web — EventoEncomendaProducer]
  |
  v
[Kafka Topic: logfast.eventos.encomenda] (3 partições, retenção 30 dias)
  |
  +---> Consumer Group: grupo-rastreamento ---> estado atual em memoria (key -> status)
  +---> Consumer Group: grupo-analytics ---> persistencia em eventos.csv
  +---> Consumer Group: grupo-dlq-demo ---> processamento com retry + DLQ

[Kafka Topic: logfast.dlq] (1 particao, retenção ilimitada — eventos inprocessaveis)

Todos os Consumer Groups consomem o mesmo tópico de forma independente.
O offset de cada grupo e gerenciado separadamente no tópico __consumer_offsets.

8.4 Estrutura do Evento

```
// Topico: logfast.eventos.encomenda
// Key: codigo da encomenda (ex: "ENC-00123") — garante ordenacao por encomenda
// Value: JSON
{
  "encomendaId": "ENC-00123",                          // identificador unico da
encomenda
  "status":      "EM_TRANSITO",                        // COLETADO | TRIAGEM |
EM_TRANSITO | ENTREGUE | FALHA
  "timestamp":   "2025-06-10T14:32:00Z",               // ISO 8601 UTC
  "cidade":      "Sao Paulo",                          // localizacao atual
  "observacao":  "Saiu do CD Guarulhos"                // descricao livre do evento
}
```

9. Tarefas da Atividade

Duracao e grupos: A atividade deve ser realizada em grupos de 2 a 3 discentes, com duracao prevista de 3 a 4 horas de laboratorio. As Tarefas 1, 2 e 3 são obrigatorias. A Tarefa 4 e classificada como desafio e possui peso adicional na avaliacao.

Tarefa 1 — Ambiente e Topologia Kafka (Obrigatoria)

Deve-se configurar o ambiente de execucao Kafka com Docker Compose e criar a topologia de tópicos necessaria para a aplicacao.

- 1.1 Criar o arquivo docker-compose.yml com o conteudo da secao 3 e executar 'docker compose up -d'. Aguardar a disponibilidade do broker verificando os logs com 'docker compose logs kafka'.
- 1.2 Acessar a Kafka-UI em http://localhost:8080 e confirmar que o broker aparece como ativo.
- 1.3 Criar o tópico logfast.eventos.encomenda com 3 partições, replication-factor 1 e retenção de 30 dias:

```
docker exec -it <container_kafka> kafka-topics \
  --create --topic logfast.eventos.encomenda \
  --partitions 3 --replication-factor 1 \
  --config retention.ms=2592000000 \
  --bootstrap-server localhost:9092
```

- 1.4 Criar o tópico logfast.dlq com 1 particao e retenção ilimitada (retention.ms=-1).
- 1.5 Documentar a topologia criada com capturas de tela da Kafka-UI exibindo os tópicos, o numero de partições e a configuracao de retenção.

Entregavel: capturas de tela da Kafka-UI demonstrando os tópicos e suas configuracoes.

Tarefa 2 — Implementacao do Producer (Obrigatoria)

Deve-se implementar o EventoEncomendaProducer (secao 5.3), com os seguintes refinamentos adicionais.

- 2.1 Criar projeto Maven conforme a secao 4.
- 2.2 Implementar o Producer com os requisitos:
  - Publicar eventos de progressao de status para 5 encomendas (ENC-001 a ENC-005), passando por todos os estagios definidos na secao 8.1.
  - Utilizar o codigo da encomenda como Key do ProducerRecord.

  - Configurar acks=all, enable.idempotence=true e linger.ms=5.
  - Incluir campo timestamp em ISO 8601 com java.time.Instant.now().
  - Registrar no console a particao e o offset de cada evento publicado via callback.

- 2.3 Executar o Producer e verificar na Kafka-UI que os eventos foram publicados. Registrar em qual particao cada encomenda (Key) foi roteada.
- 2.4 Elaborar uma explicacao escrita de por que eventos da mesma encomenda são sempre roteados para a mesma particao, e o que ocorreria se a Key fosse null.

Entregavel: codigo-fonte + captura da Kafka-UI com os eventos publicados + explicacao escrita.

Tarefa 3 — Consumer Groups e Commit Manual (Obrigatoria)

Devem ser implementados dois Consumer Groups independentes consumindo o mesmo tópico de forma autonoma.

- 3.1 Consumer Group grupo-rastreamento (baseado no RastreamentoConsumer da secao 6.4):
  - Manter um Map<String, String> com o ultimo status de cada encomenda.
  - Imprimir o estado consolidado apos cada lote: 'ENC-001 -> ENTREGUE'.
  - Utilizar commitSync() manual apos cada lote processado.

- 3.2 Consumer Group grupo-analytics (baseado no AnalyticsConsumer da secao 6.5):
  - Persistir todos os eventos em arquivo CSV: encomendaId, status, timestamp.
  - Realizar writer.flush() antes de commitSync() — justificar essa ordem no relatorio.

- 3.3 Executar os dois consumers simultaneamente em terminais distintos. Verificar na Kafka-UI (aba Consumer Groups) que cada grupo mantem lag e offsets independentes.
- 3.4 Apos os consumers terminarem de processar todos os eventos, reiniciar o grupo-analytics com a propriedade auto.offset.reset=earliest. Registrar e analisar o comportamento observado e suas implicacoes em ambiente de producao.

Entregavel: codigo-fonte dos dois consumers + arquivo CSV gerado + analise escrita do comportamento.

Tarefa 4 — Retry e Dead Letter Queue (Desafio)

Deve-se implementar o ConsumerComDLQ (secao 7.2) e demonstrar o funcionamento do mecanismo de retry e encaminhamento para DLQ.

- 4.1 Implementar o ConsumerComDLQ conforme o exemplo da secao 7.2.
- 4.2 Garantir que o tópico logfast.dlq foi criado com as configuracoes da Tarefa 1.4.
- 4.3 Executar o consumer e verificar que:
  - Eventos com status TRIAGEM falham nas 3 tentativas e são publicados no tópico logfast.dlq.
  - Os headers motivo e tópico-origem estao presentes nos eventos da DLQ (verificar na Kafka-UI).
  - O consumer continua processando os eventos subsequentes normalmente apos o encaminhamento para DLQ.

- 4.4 Implementar um segundo consumer (grupo grupo-dlq-monitor) que consome o tópico logfast.dlq e imprime um relatorio com: key, motivo do encaminhamento e timestamp.
- 4.5 Elaborar uma analise escrita respondendo: por que o commitSync() e chamado mesmo para eventos encaminhados para a DLQ? Qual seria o problema de não chamar o commit nesses casos?

Entregavel: codigo-fonte das duas classes + captura da Kafka-UI mostrando eventos no tópico DLQ + analise escrita.

10. Questoes de Analise e Reflexao

As questoes a seguir devem ser respondidas individualmente como parte do relatorio de entrega. As respostas devem ser fundamentadas nos conceitos apresentados e na experiencia pratica obtida durante a execucao das tarefas.

6. Por que o Kafka garante a ordenacao de eventos de uma mesma encomenda quando a Key e utilizada corretamente? O que aconteceria com a ordenacao caso a Key fosse null em todos os ProducerRecords?

7. Na Tarefa 3, ao reiniciar o grupo-analytics com auto.offset.reset=earliest, todos os eventos historicos foram reprocessados. Esse comportamento seria aceitavel em producao? Quais estrategias permitem controlar o ponto de retomada em uma reimplantacao?

8. Explique a diferenca entre commitSync() e commitAsync(). Em qual situacao o uso de commitAsync() seria preferivel ao de commitSync()? Quais cuidados são necessarios ao utiliza-lo?

9. Na Tarefa 4, o commitSync() e chamado mesmo para eventos encaminhados para a DLQ. Por que essa e a abordagem correta? O que aconteceria se o commit não fosse realizado nesses casos?

10. Considere um tópico com 3 partições e um Consumer Group com 5 instancias do mesmo consumer. Como o Kafka distribui as partições entre as instancias? Qual e o impacto operacional de ter mais instancias do que partições?

11. Criterios de Avaliacao

Criterio	Pontos	Indicadores de Qualidade
Tarefa 1 — Topologia Kafka configurada	15	Topicos criados com partições e retenção corretas; capturas da Kafka-UI
Tarefa 2 — Producer com Keys, garantias e callback	20	Codigo funcional; idempotencia; linter.ms; analise do roteamento por particao
Tarefa 3 — Consumer Groups independentes e commit manual	25	Dois grupos com offsets independentes; CSV gerado; analise do reprocessamento
Tarefa 4 — Retry e DLQ (Desafio)	25	Retry funcional; headers corretos na DLQ; consumer monitor; analise do commitSync
Questoes de analise e reflexao	15	Precisao conceitual; fundamentacao nos exemplos; clareza e profundidade
TOTAL	100	

Instrucoes de entrega: O projeto Maven, os arquivos gerados (CSV, logs) e o relatorio em PDF (capturas de tela e respostas as questoes) devem ser comprimidos em um arquivo .zip nomeado no formato grupo_<nomes>.zip e submetidos pelo ambiente de gerenciamento da disciplina ate a data estabelecida pelo docente.

12. Referencia — Comandos CLI Kafka

Os comandos a seguir são executados dentro do container Kafka via 'docker exec -it <container> <comando>':

Gerenciamento de Topicos

```
# Listar todos os tópicos
kafka-topics --list --bootstrap-server localhost:9092

# Criar tópico
kafka-topics --create --topic <nome> \
  --partitions <N> --replication-factor 1 \
  --bootstrap-server localhost:9092

# Descrever tópico (partições, replicas, ISR, configuracoes)
kafka-topics --describe --topic <nome> --bootstrap-server localhost:9092

# Alterar configuracao de retenção
kafka-configs --alter --entity-type topics --entity-name <nome> \
  --add-config retention.ms=2592000000 \
  --bootstrap-server localhost:9092
```

Producao e Consumo de Teste

```
# Producer de teste (digitar JSON no terminal)
kafka-console-producer --topic <nome> \
  --property key.separator=: --property parse.key=true \
  --bootstrap-server localhost:9092

# Consumer de teste a partir do inicio
kafka-console-consumer --topic <nome> \
  --from-beginning --property print.key=true \
  --bootstrap-server localhost:9092
```

Gerenciamento de Consumer Groups

```
# Listar todos os Consumer Groups
kafka-consumer-groups --list --bootstrap-server localhost:9092

# Verificar offsets e lag de um grupo
kafka-consumer-groups --describe --group <group.id> \
  --bootstrap-server localhost:9092

# Resetar offset para o inicio (usar com cautela em producao)
kafka-consumer-groups --reset-offsets --group <group.id> \
  --topic <nome> --to-earliest --execute \
  --bootstrap-server localhost:9092

# Deletar um Consumer Group (group deve estar inativo)
kafka-consumer-groups --delete --group <group.id> \
  --bootstrap-server localhost:9092
```

Referencias

- Apache Kafka Documentacao: kafka.apache.org/documentation
- kafka-clients 3.7.0 Javadoc: javadoc.io/doc/org.apache.kafka/kafka-clients/3.7.0
- Kafka-UI: github.com/provectus/kafka-ui
- NARKHEDE, N. et al. Kafka: The Definitive Guide. 2. ed. O'Reilly, 2021.
- KLEPPMANN, M. Designing Data-Intensive Applications. O'Reilly, 2017.

Em caso de duvidas, consulte o docente responsavel ou registre uma issue no repositorio da disciplina.